JavaScript is disabled on your browser.

**Java SE 27 & JDK 27**

# Interface AnnotatedElement

**All Known Subinterfaces:**

`AnnotatedArrayType`, `AnnotatedParameterizedType`, `AnnotatedType`, `AnnotatedTypeVariable`, `AnnotatedWildcardType`, `GenericDeclaration`, `TypeVariable<D>`

**All Known Implementing Classes:**

`AccessibleObject`, `Class`, `Constructor`, `Executable`, `Field`, `Method`, `Module`, `Package`, `Parameter`, `RecordComponent`

---

`public interface` **`AnnotatedElement`**

Represents an annotated construct of the program currently running in this VM. A construct is either an element or a type. Annotations on an element are on a *declaration*, whereas annotations on a type are on a specific *use* of a type name. As defined by *The Java Language Specification* section §9.7.4, an annotation on an element is a *declaration annotation* and an annotation on a type is a *type annotation*. Note that any annotations returned by methods on the `AnnotatedType` interface and its subinterfaces are type annotations as the entity being potentially annotated is a type. Annotations returned by methods outside of the `AnnotatedType` hierarchy are declaration annotations.

This interface allows annotations to be read reflectively. All annotations returned by methods in this interface are immutable and serializable. The arrays returned by methods of this interface may be modified by callers without affecting the arrays returned to other callers.

The `getAnnotationsByType(Class)` and `getDeclaredAnnotationsByType(Class)` methods support multiple annotations of the same type on an element. If the argument to either method is a repeatable annotation type (JLS §9.6), then the method will "look through" a container annotation (JLS §9.7), if present, and return any annotations inside the container. Container annotations may be generated at compile-time to wrap multiple annotations of the argument type.

The terms *directly present*, *indirectly present*, *present*, and *associated* are used throughout this interface to describe precisely which annotations are returned by methods:

- An annotation $A$ is *directly present* on an element $E$ if $E$ has a `RuntimeVisibleAnnotations` or `RuntimeVisibleParameterAnnotations` or `RuntimeVisibleTypeAnnotations` attribute, and the attribute contains $A$.
- An annotation $A$ is *indirectly present* on an element $E$ if $E$ has a `RuntimeVisibleAnnotations` or `RuntimeVisibleParameterAnnotations` or `RuntimeVisibleTypeAnnotations` attribute, and $A$ 's type is repeatable,

and the attribute contains exactly one annotation whose value element contains *A* and whose type is the containing annotation type of *A* 's type.

- An annotation *A* is *present* on an element *E* if either:
  - *A* is directly present on *E*; or
  - No annotation of *A* 's type is directly present on *E*, and *E* is a class, and *A* 's type is inheritable, and *A* is present on the superclass of *E*.
- An annotation *A* is *associated* with an element *E* if either:
  - *A* is directly or indirectly present on *E*; or
  - No annotation of *A* 's type is directly or indirectly present on *E*, and *E* is a class, and *A*'s type is inheritable, and *A* is associated with the superclass of *E*.

The table below summarizes which kind of annotation presence different methods in this interface examine.

**Overview of kind of presence detected by different AnnotatedElement methods**

| **Method** | | **Kind of Presence** | | | |
|---|---|---|---|---|---|
| **Return Type** | **Signature** | **Directly Present** | **Indirectly Present** | **Present** | **A** |
| `T` | `getAnnotation(Class<T>)` | | | X | |
| `Annotation[]` | `getAnnotations()` | | | X | |
| `T[]` | `getAnnotationsByType(Class<T>)` | | | | |
| `T` | `getDeclaredAnnotation(Class<T>)` | X | | | |
| `Annotation[]` | `getDeclaredAnnotations()` | X | | | |
| `T[]` | `getDeclaredAnnotationsByType(Class<T>)` | X | X | | |

For an invocation of `get[Declared]AnnotationsByType(Class <T>)`, the order of annotations which are directly or indirectly present on an element *E* is computed as if indirectly present annotations on *E* are directly present on *E* in place of their container annotation, in the order in which they appear in the value element of the container annotation.

There are several compatibility concerns to keep in mind if an annotation type *T* is originally *not* repeatable and later modified to be repeatable. The containing annotation type for *T* is *TC*.

- Modifying *T* to be repeatable is source and binary compatible with existing uses of *T* and with existing uses of *TC*. That is, for source compatibility, source code with annotations of type *T* or of type *TC* will still compile. For binary compatibility, class files with annotations of type *T* or of type *TC* (or with other kinds of uses of type *T* or of type *TC*) will link against the modified version of *T* if they linked against the earlier version. (An annotation type *TC* may informally serve as an acting containing annotation type before *T* is modified to be formally repeatable.

Alternatively, when *T* is made repeatable, *TC* can be introduced as a new type.)

- If an annotation type *TC* is present on an element, and *T* is modified to be repeatable with *TC* as its containing annotation type then:
    - The change to *T* is behaviorally compatible with respect to the `get[Declared]Annotation(Class<T>)` (called with an argument of *T* or *TC*) and `get[Declared]Annotations()` methods because the results of the methods will not change due to *TC* becoming the containing annotation type for *T*.
    - The change to *T* changes the results of the `get[Declared]AnnotationsByType(Class<T>)` methods called with an argument of *T*, because those methods will now recognize an annotation of type *TC* as a container annotation for *T* and will "look through" it to expose annotations of type *T*.
- If an annotation of type *T* is present on an element and *T* is made repeatable and more annotations of type *T* are added to the element:
    - The addition of the annotations of type *T* is both source compatible and binary compatible.
    - The addition of the annotations of type *T* changes the results of the `get[Declared]Annotation(Class<T>)` methods and `get[Declared]Annotations()` methods, because those methods will now only see a container annotation on the element and not see an annotation of type *T*.
    - The addition of the annotations of type *T* changes the results of the `get[Declared]AnnotationsByType(Class<T>)` methods, because their results will expose the additional annotations of type *T* whereas previously they exposed only a single annotation of type *T*.

If an annotation returned by a method in this interface contains (directly or indirectly) a `Class`-valued member referring to a class that is not accessible in this VM, attempting to read the class by calling the relevant Class-returning method on the returned annotation will result in a `TypeNotPresentException`.

Similarly, attempting to read an enum-valued member will result in a `EnumConstantNotPresentException` if the enum constant in the annotation is no longer present in the enum class.

If an annotation type *T* is (meta-)annotated with an `@Repeatable` annotation whose value element indicates a type *TC*, but *TC* does not declare a `value()` method with a return type of *T*`[]`, then an exception of type `AnnotationFormatError` is thrown.

Finally, attempting to read a member whose definition has evolved incompatibly will result in an `AnnotationTypeMismatchException` or an `IncompleteAnnotationException`.

**Since:**

1.5

**See Also:**

EnumConstantNotPresentException, TypeNotPresentException, AnnotationFormatError, AnnotationTypeMismatchException, IncompleteAnnotationException

## *Method Summary*

| All Methods | Instance Methods | |
|---|---|---|

| Modifier and Type | Method | Description |
|---|---|---|
| <T extends Annotation> T | **getAnnotation**(Class<T> annotationClas | Returns this element's annotation for the specified type if such an annotation is *present*, else null. |
| Annotation[] | **getAnnotations**() | Returns annotations that are *present* on this element. |
| default <T extends Annotation> T[] | **getAnnotationsByType**(Class<T> annotationClas | Returns annotations that are *associated* with this element. |
| default <T extends Annotation> T | **getDeclaredAnnotation**(Class<T> annotationClas | Returns this element's annotation for the specified type if such an annotation is *directly present*, else null. |
| Annotation[] | **getDeclaredAnnotations**() | Returns annotations that are *directly present* on this element. |
| default <T extends Annotation> T[] | **getDeclaredAnnotationsBy**(Class<T> annotationClas | Returns this element's annotation(s) for the specified type if such annotations are either *directly present* or *indirectly present*. |

| | | |
|---|---|---|
| `default boolean` | **`isAnnotationPresent`**`(Class<? extends Annotation> annotationCl` | Returns true if an annotation for the specified type is *present* on this element, else false. |

## *Method Details*

### isAnnotationPresent

```
default boolean isAnnotationPresent
(Class<? extends Annotation> annotationClass)
```

Returns true if an annotation for the specified type is *present* on this element, else false. This method is designed primarily for convenient access to marker annotations.

The truth value returned by this method is equivalent to:
`getAnnotation(annotationClass) != null`

**Implementation Requirements:**

The default implementation returns `getAnnotation(annotationClass) != null`.

**Parameters:**

`annotationClass` - the Class object corresponding to the annotation type

**Returns:**

true if an annotation for the specified annotation type is present on this element, else false

**Throws:**

`NullPointerException` - if the given annotation class is null

### getAnnotation

```
<T extends Annotation> T getAnnotation(Class<T> annotationClass)
```

Returns this element's annotation for the specified type if such an annotation is *present*, else null.

**Type Parameters:**

`T` - the type of the annotation to query for and return if present

**Parameters:**

`annotationClass` - the Class object corresponding to the annotation type

**Returns:**

this element's annotation for the specified annotation type if present on this element, else null

**Throws:**

`NullPointerException` - if the given annotation class is null

## getAnnotations

```
Annotation[] getAnnotations()
```

Returns annotations that are *present* on this element. If there are no annotations *present* on this element, the return value is an array of length 0. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

**Returns:**

annotations present on this element

## getAnnotationsByType

```
default <T extends Annotation> T[] getAnnotationsByType
(Class<T> annotationClass)
```

Returns annotations that are *associated* with this element. If there are no annotations *associated* with this element, the return value is an array of length 0. The difference between this method and `getAnnotation(Class)` is that this method detects if its argument is a *repeatable annotation type* (JLS §9.6), and if so, attempts to find one or more annotations of that type by "looking through" a container annotation. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

**Implementation Requirements:**

The default implementation first calls `getDeclaredAnnotationsByType(Class)` passing `annotationClass` as the argument. If the returned array has length greater than zero, the array is returned. If the returned array is zero-length and this `AnnotatedElement` is a class and the argument type is an inheritable annotation type, and the superclass of this `AnnotatedElement` is non-null, then the returned result is the result of calling `getAnnotationsByType(Class)` on the superclass with `annotationClass` as the argument. Otherwise, a zero-length array is returned.

**Type Parameters:**

`T` - the type of the annotation to query for and return if present

**Parameters:**

`annotationClass` - the Class object corresponding to the annotation type

**Returns:**

all this element's annotations for the specified annotation type if associated with this element, else an array of length zero

**Throws:**

`NullPointerException` - if the given annotation class is null

**Since:**

1.8

## getDeclaredAnnotation

```
default <T extends Annotation> T getDeclaredAnnotation
(Class<T> annotationClass)
```

Returns this element's annotation for the specified type if such an annotation is *directly present*, else null. This method ignores inherited annotations. (Returns null if no annotations are directly present on this element.)

**Implementation Requirements:**

The default implementation first performs a null check and then loops over the results of `getDeclaredAnnotations()` returning the first annotation whose annotation type matches the argument type.

**Type Parameters:**

`T` - the type of the annotation to query for and return if directly present

**Parameters:**

`annotationClass` - the Class object corresponding to the annotation type

**Returns:**

this element's annotation for the specified annotation type if directly present on this element, else null

**Throws:**

`NullPointerException` - if the given annotation class is null

**Since:**

1.8

## getDeclaredAnnotationsByType

```
default <T extends Annotation> T[] getDeclaredAnnotationsByType
(Class<T> annotationClass)
```

Returns this element's annotation(s) for the specified type if such annotations are either *directly present* or *indirectly present*. This method ignores inherited annotations. If there are no specified annotations directly or indirectly present on this element, the return value is an array of length 0. The difference between this method and `getDeclaredAnnotation(Class)` is

that this method detects if its argument is a *repeatable annotation type* (JLS §9.6), and if so, attempts to find one or more annotations of that type by "looking through" a container annotation if one is present. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

**Implementation Requirements:**

The default implementation may call `getDeclaredAnnotation(Class)` one or more times to find a directly present annotation and, if the annotation type is repeatable, to find a container annotation. If annotations of the annotation type `annotationClass` are found to be both directly and indirectly present, then `getDeclaredAnnotations()` will get called to determine the order of the elements in the returned array.

Alternatively, the default implementation may call `getDeclaredAnnotations()` a single time and the returned array examined for both directly and indirectly present annotations. The results of calling `getDeclaredAnnotations()` are assumed to be consistent with the results of calling `getDeclaredAnnotation(Class)`.

**Type Parameters:**

`T` - the type of the annotation to query for and return if directly or indirectly present

**Parameters:**

`annotationClass` - the Class object corresponding to the annotation type

**Returns:**

all this element's annotations for the specified annotation type if directly or indirectly present on this element, else an array of length zero

**Throws:**

`NullPointerException` - if the given annotation class is null

**Since:**

1.8

## getDeclaredAnnotations

`Annotation[] getDeclaredAnnotations()`

Returns annotations that are *directly present* on this element. This method ignores inherited annotations. If there are no annotations *directly present* on this element, the return value is an array of length 0. The caller of this method is free to modify the returned array; it will have no effect on the arrays returned to other callers.

**Returns:**

annotations directly present on this element

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.